

Tema 4. Compresión de Datos

José A. Montenegro

Dpto. Lenguajes y Ciencias de la Computación
ETSI Informática. Universidad de Málaga
jmmontes@uma.es

5 de febrero de 2026

- 1 Codificación en bloques
- 2 Distribuciones del producto de conjuntos
- 3 Fuentes Estacionarias
 - Codificación de un código estacionario
- 4 Algoritmos para compresión de los datos
 - Codificación Aritmética
 - Utilizar números como palabras codificadas
 - Codificación con un diccionario dinámico

Sección 1

Codificación en bloques

- Hemos formalizado la codificación como una función $c : S \rightarrow T^*$ que reemplaza los símbolos de un alfabeto S por cadenas de símbolos en un Alfabeto T .
- Aunque inicialmente hemos establecido los elementos de S como **objetos simples**, tales como letras de un alfabeto natural, no hay necesidad lógica para esta restricción.
- En la práctica, a menudo es útil dividir la cadena de símbolos emitido por una fuente en **bloques** y tratar a los bloques creados como símbolos.
- Por ejemplo, supongamos que una fuente emite una cadena de letras, con las letras x,y o z, por lo que una cadena típica podría ser:

yzxxxzyxzyxzzzyxxzyzxxzzyyy...

- En este ejemplo el alfabeto S es el conjunto $\{x,y,z\}$. Por otro lado, podemos dividir la cadena en bloques de longitud 2:

yz xx zy xz yx zz yx xz yz xz zy yy ...

- Ahora el alfabeto es el conjunto S^2 de pares ordenados:

$$S^2 = \{xx, xy, xz, yx, yy, yz, zx, zy, zz\}.$$

- Anteriormente, hemos considerado el problema de codificar la fuente (S,p) para que la longitud media de palabra L sea tan pequeña como sea posible.
- Hemos hallado que la entropía $H_b(p)$ es el límite inferior para L , pero este límite es rara vez alcanzado.
- Ahora explicaremos como codificar en bloques nos permite acercarnos más fácilmente al límite inferior.

Ejemplo 1

Consideremos un escáner que procesa un documento en blanco y negro. Supongamos que emite B (pixel blanco) y N (pixel negro) con probabilidades $p_B = 0,9$ y $p_N = 0,1$. Calcula la entropía de esta distribución, asumiendo que la fuente es sin memoria, encontrando los mejores código binarios para utilizar bloques de tamaño 1, 2 y 3.

Ejemplo 1

Consideremos un escáner que procesa un documento en blanco y negro. Supongamos que emite B (pixel blanco) y N (pixel negro) con probabilidades $p_B = 0,9$ y $p_N = 0,1$. Calcula la entropía de esta distribución, asumiendo que la fuente es sin memoria, encontrando los mejores código binarios para utilizar bloques de tamaño 1, 2 y 3.

Solución:

La entropía es:

$$H(p) = 0,9 \times \log_2(1/0,9) + 0,1 \times \log_2(1/0,1) \approx 0,469.$$

- Utilizando **bloques de tamaño 1**, la mejor forma para codificar la fuente es $B \mapsto 0, N \mapsto 1$ debido a que otro código debería utilizar una codificación mayor que 1.
 - ▶ Por tanto, este código tiene una longitud media de palabra $L_1 = 1$, el cual es peor que el límite inferior establecido en la entropía 0.469.
 - ▶ Un documento que contiene N pixels es codificado como una cadena de N bits, aunque desde el punto de vista teórica, la cantidad de información es equivalente solamente a **0.469N** bits.
- ... continua

- Consideramos que ocurre cuando dividimos el mensaje en **bloques de tamaño 2**. Ahora tenemos una fuente con 4 símbolos, y las probabilidades son:

BB	BN	NB	NN
0.81	0.09	0.09	0.01

- ¿Cual es el código binario óptimo para esta fuente?. Una aplicación de la regla de Huffman nos da el código

$$BB \mapsto 0, BN \mapsto 10, NB \mapsto 110, NN \mapsto 111$$

- La longitud media de este código es:

$$L_2 = 0,81 \times 1 + 0,09 \times 2 + (0,09 + 0,01) \times 3 = 1,29.$$

- ▶ Obsérvese que un símbolo ahora representa dos pixels.
 - ▶ Un documento con N pixels tendrá $\frac{N}{2}$ símbolos y será codificado por una cadena con $L_2 \times \frac{N}{2} = 0.645N$ bits.
 - ▶ Esto muestra que tenemos una significativa mejora de nuestro primer intento de alcanzar el límite teórico de $0.469N$.
- ... continua

- ¿Qué ocurre si utilizamos **bloques de longitud 3**?. Ahora la fuente tiene 8 símbolos y las probabilidades son:

BBB	BBN	BNB	BNN	NBB	NBN	NNB	NNN
0.729	0.081	0.081	0.009	0.081	0.009	0.009	0.001

- Otra aplicación de la regla de Huffman produce el código:

$$\begin{aligned}
 BBB &\mapsto 0, BBN \mapsto 100, BNB \mapsto 101, BNN \mapsto 1110, \\
 NBB &\mapsto 111, NBN \mapsto 11001, NNB \mapsto 11010, NNN \mapsto 11011
 \end{aligned}$$

- La longitud media es :

$$L_3 = 0,729 \times 1 + 0,081 \times 3 + 0,081 \times 3 + 0,009 \times 4 + 0,081 \times 3 + 0,009 \times 5 + 0,009 \times 5 + 0,001 \times 5 = 0,729 + 0,729 + 0,036 + 0,09 + 0,005 = 1,589.$$

- Repitiendo el argumento anterior, veremos que un documento que contiene N pixels puede ser codificado por una cadena de $L_3 \frac{N}{3} = 0.529 N$ bits, la cual es más cercana al límite inferior teórico de $0.469N$.

La técnica descrita en el ejemplo es la base de la compresión de los datos. En adelante exploraremos las bases teóricas y describiremos algunas de las reglas de codificación que pueden ser utilizadas para implementarla.

Ejercicio 1

Considere una fuente sin memoria que emite los símbolos A y B con probabilidades $p_A = 0,8, p_B = 0,2$. Calcule la entropía para esta fuente. Encuentra los códigos Huffman binarios para las fuentes utilizando bloques de tamaño 2 y 3, y calcula la longitud media de palabra L_2 y L_3 .

Ejercicio 1

Considere una fuente sin memoria que emite los símbolos A y B con probabilidades $p_A = 0,8, p_B = 0,2$. Calcule la entropía para esta fuente. Encuentra los códigos Huffman binarios para las fuentes utilizando bloques de tamaño 2 y 3, y calcula la longitud media de palabra L_2 y L_3 .

Solución:

- La entropía de la distribución dada es:

$$H_2(p) = 0,2 \times \log_2\left(\frac{1}{0,2}\right) + 0,8 \times \log_2\left(\frac{1}{0,8}\right) \simeq 0,72$$

- La codificación obvia es $A \mapsto 0, B \mapsto 1$, y con $L_1 = 1$.

- Para bloques de longitud 2 las probabilidades son:

$$AA = 0.64, AB = 0.16, BA = 0.16, BB = 0.04$$

- y aplicando la regla de Huffman tendríamos:

$$AA \mapsto 0, AB \mapsto 10, BA \mapsto 110, BB \mapsto 111$$

$$L_2 = 0,64 \times 1 + 0,16 \times 2 + 0,16 \times 3 + 0,04 \times 3 = 1,56 \text{ y } \frac{L_2}{2} = 0,78.$$

- ... continua

- Para bloques de longitud 3 las probabilidades son:

AAA	AAB	ABA	BAA	ABB	BAB	BBA	BBB
0.512	0.128	0.128	0.128	0.032	0.032	0.032	0.008
0	100	101	110	11100	11101	11110	11111

$$L_3 = 0,512 \times 1 + 0,128 \times 3 + 0,128 \times 3 + 0,128 \times 3 + 0,032 \times 5 + 0,032 \times 5 + 0,032 \times 5 + 0,008 \times 5 \simeq 2,18 \text{ y}$$

$$\frac{L_3}{3} = 0,728.$$

- Desde la observación podemos intuir que el límite L_n/n cuando $n \rightarrow \infty$ es la entropía, 0.72.

Sección 2

Distribuciones del producto de conjuntos

Distribuciones del producto de conjuntos

- Sea $S' = \{s'_1, s'_2, \dots, s'_m\}$, $S'' = \{s''_1, s''_2, \dots, s''_n\}$, y sea $Y = S' \times S''$ el producto de los conjuntos, que contiene los pares de símbolos $s'_i s''_j$.
- Sea p la **probabilidad de distribución sobre Y** , denotamos la probabilidad de $s'_i s''_j$ por p_{ij} .

Definición 1 (Distribución Marginal, independencia)

Utilizando la notación anterior, sea

$$p'_i = \sum_{j=1}^n p_{ij} \quad (i = 1, 2, \dots, m), \quad p''_j = \sum_{i=1}^m p_{ij} \quad (j = 1, 2, \dots, n)$$

- ▶ La distribución p' sobre S' y p'' en S'' son conocidos como las **distribuciones marginales** asociadas con p .
- ▶ p'_i es la probabilidad que el primer componente es s'_i , y p''_j es la probabilidad que el segundo componente es s''_j .
- ▶ Las distribuciones p' y p'' son **independientes** si $p_{ij} = p'_i \times p''_j$.

Ejemplo 2

Supongamos que $S' = \{a, b\}$ y $S'' = \{c, d\}$ y la distribución p es dada por la siguiente tabla. (Por ejemplo, la entrada en la fila a y la columna d es p_{ad}).

	c	d
a	0.3	0.1
b	0.4	0.2

Encontrar las distribuciones marginales p' y p'' . ¿Son las distribuciones independientes?

Ejemplo 2

Supongamos que $S' = \{a, b\}$ y $S'' = \{c, d\}$ y la distribución p es dada por la siguiente tabla. (Por ejemplo, la entrada en la fila a y la columna d es p_{ad}).

	c	d
a	0.3	0.1
b	0.4	0.2

Encontrar las distribuciones marginales p' y p'' . ¿Son las distribuciones independientes?

Solución:

$$p'_a = p_{ac} + p_{ad} = 0,4, \quad p'_b = p_{bc} + p_{bd} = 0,6$$

$$p''_c = p_{ac} + p_{bc} = 0,7, \quad p''_d = p_{ad} + p_{bd} = 0,3$$

Las distribuciones no son independiente debido (por ejemplo):

$$p_{ac} = 0,3 \text{ mientras } p'_a p''_c = 0,28.$$

Teorema 1

Las entropías de las distribuciones p , p' , p'' satisfacen

$$H(p) \leq H(p') + H(p'')$$

que cumple la igualdad sii p' y p'' son independientes.

- Resultados similares pueden establecerse para un producto de más de dos conjuntos.
- Nuestra principal interés recae en el caso cuando todos los conjuntos son el mismo (S):
 - ▶ con el producto S^r de $r \geq 2$ copias de un conjunto dado S .
 - ▶ en ese caso un elemento de S^r es solamente una palabra de longitud r en el alfabeto S .

Ejemplo 3

Supongamos que una fuente emite una cadena de bits, y un número de observaciones sugiere que las frecuencias de bloques de longitud 2 son dados por la siguiente distribución de probabilidad p en $\mathbb{B}^2$. Por ejemplo, $p_{01} = 0,4$ significa que sobre el 40 de cada 100 bloques son 01.

	0	1
0	0.1	0.4
1	0.4	0.1

Encuentra las distribuciones marginales p' y p'' , calcula sus entropías y verifica que el teorema 1 se cumple.

Ejemplo 3

Supongamos que una fuente emite una cadena de bits, y un número de observaciones sugiere que las frecuencias de bloques de longitud 2 son dados por la siguiente distribución de probabilidad p en $\mathbb{B}^2$. Por ejemplo, $p_{01} = 0,4$ significa que sobre el 40 de cada 100 bloques son 01.

	0	1
0	0.1	0.4
1	0.4	0.1

Encuentra las distribuciones marginales p' y p'' , calcula sus entropías y verifica que el teorema 1 se cumple.

Solución:

- Las distribuciones marginales son $p' = [0.5, 0.5]$ y $p'' = [0.5, 0.5]$.
- $H(p') = H(p'') = H(0.5) = 1$.
- $H(p) = 0,1 \times \log(1/0,1) + 0,4 \times \log(1/0,4) + 0,4 \times \log(1/0,4) + 0,1 \times \log(1/0,1) \approx 1,722$
- Teorema 1 $H(p) \leq H(p') + H(p'') = 1,72 \leq 1 + 1$ se cumple.
- Las distribuciones marginales p' y p'' son las mismas, pero **no son independientes** ya que, por ejemplo $p_{00} = 0,1$ mientras que $p'_0 \times p''_0 = 0,25 \rightarrow$ la fuente no es sin memoria.

Ejercicio 2

Supongamos que $X' = \{u, v, w\}$ y $X'' = \{y, z\}$ y la distribución p en $X' \times X''$ es dada por la siguiente tabla.

	y	z
u	0.2	0.1
v	0.3	0.1
w	0.1	0.2

¿Son las distribuciones p' y p'' independientes?

Ejercicio 2

Supongamos que $X' = \{u,v,w\}$ y $X'' = \{y,z\}$ y la distribución p en $X' \times X''$ es dada por la siguiente tabla.

	y	z
u	0.2	0.1
v	0.3	0.1
w	0.1	0.2

¿Son las distribuciones p' y p'' independientes?

Solución:

$$p'_u = p_{uy} + p_{uz} = 0,3, \quad p'_v = p_{vy} + p_{vz} = 0,4, \quad p'_w = p_{wy} + p_{wz} = 0,3$$

$$p''_y = p_{uy} + p_{vy} + p_{wy} = 0,6, \quad p''_z = p_{uz} + p_{vz} + p_{wz} = 0,4$$

Las distribuciones **no** son independiente debido (por ejemplo):

$$p_{uy} = 0,2 \text{ mientras } p'_u \times p''_y = 0,3 * 0,6 = 0,18.$$

Ejercicio 3

Verifica que las entropías de las distribuciones definidas en el ejercicio anterior satisfacen el teorema 1

Ejercicio 3

Verifica que las entropías de las distribuciones definidas en el ejercicio anterior satisfacen el teorema 1

Solución:

	y	z	
u	0.2	0.1	$p'=[0.3,0.4,0.3]; p''=[0.6,0.4];$
v	0.3	0.1	
w	0.1	0.2	

Teorema 1: $H(p) \leq H(p') + H(p'') = 2,45 \leq 1,57 + 0,97 = \mathbf{2,45} \leq \mathbf{2,54}$

- $H(p') = 0,3 \times \log_2(1/0,3) + 0,4 \times \log_2(1/0,4) + 0,3 \times \log_2(1/0,3) \simeq \mathbf{1,57}$
- $H(p'') = 0,6 \times \log_2(1/0,6) + 0,4 \times \log_2(1/0,4) \simeq \mathbf{0,97}$
- $H(p) = 0,2 \times \log_2(1/0,2) + 0,1 \times \log_2(1/0,1) + 0,3 \times \log_2(1/0,3) + 0,1 \times \log_2(1/0,1) + 0,1 \times \log_2(1/0,1) + 0,2 \times \log_2(1/0,2) \simeq \mathbf{2,45}$

Ejercicio 4

Hemos observado que un cierto flujo de bits, la frecuencia de bloques de longitud 2, son dados por la siguiente distribución de probabilidad en $\mathbb{B}^2$.

	0	1
0	0.35	0.15
1	0.15	0.35

Establezca las distribuciones marginales p' y p'' , calcule sus entropías, y verifique que se cumpla el teorema 1.

Ejercicio 4

Hemos observado que un cierto flujo de bits, la frecuencia de bloques de longitud 2, son dados por las siguiente distribución de probabilidad en $\mathbb{B}^2$.

	0	1
0	0.35	0.15
1	0.15	0.35

Establezca las distribuciones marginales p' y p'' , calcula sus entropías, y verifica que se cumpla el teorema 1.

Solución:

Teorema 1: $H(p) \leq H(p') + H(p'') = 1,88 \leq 1 + 1 = \mathbf{1,88} \leq \mathbf{2}$

p' y $p'' = 0.5$; $H(p') = H(p'') = 1$

$H(p) = 0,35 \times \log_2(1/0,35) + 0,15 \times \log_2(1/0,15) + 0,15 \times \log_2(1/0,35) + 0,15 \times \log_2(1/0,15) \simeq \mathbf{1,88}$

Sección 3

Fuentes Estacionarias

Fuentes Estacionarias

- La cadena emitida por una fuente $\xi_1\xi_2\xi_3\dots$ como una secuencia de variables aleatorias idénticamente distribuidas $\xi_k (k = 1, 2, 3\dots)$, tomando valores en un conjunto S .
- Consideramos una cadena emitida por una fuente como **una cadena de bloques**, donde cada bloque es una variable aleatoria que toma valores en el **conjunto S^r** de cadenas de longitud r .
 - ▶ Tomando $r=2$ tendríamos una cadena de variables aleatorias $\xi_{2k-1}\xi_{2k}$.
 - ▶ Podríamos definir una distribución de probabilidad p^2 en S^2 mediante la siguiente regla:

$$p^2(s_i s_j) = Pr(\xi_{2k-1}\xi_{2k} = s_i s_j) = Pr(\xi_{2k-1} = s_i, \xi_{2k} = s_j)$$

- Debemos asumir que la probabilidad de emitir el par de símbolos $s_i s_j$ **no depende de k , la posición en la cadena**. *Por ejemplo, para calcular $p^2(cb)$ será equivalente en $aaaaacb$ y $cbaaaaa$.*

Definición 2 (Fuente estacionaria)

Una fuente que emite una cadena $\xi_1\xi_2\xi_3\dots$ es **estacionaria** si, para cualquier entero positivo $l_1, l_2, \dots, l_r$ la probabilidad:

$$P^r(\xi_{k+l_1} = x_1, \xi_{k+l_2} = x_2, \dots, \xi_{k+l_r} = x_r)$$

depende solamente de la cadena $x_1, x_2 \dots x_r$ no de k .

- Usualmente consideramos los símbolos consecutivos, lo que sería el caso $l_1 = 1, l_2 = 2, \dots, l_r = r$ de la definición.
- Una **fuentes sin memoria** es un caso muy **especial de una fuente estacionaria**. En tal caso, cada p^r es determinado por p^1 :

$$p^r(x_1x_2\dots x_r) = p^1(x_1) \times p^1(x_2) \times \dots \times p^1(x_r)$$

Ejercicio 5

Considérese una fuente que emite símbolos de un alfabeto $S = \{a, b, c\}$, con la probabilidad de distribución p^2 en S^2 definida por la siguiente tabla:

	a	b	c
a	0.39	0.17	0.04
b	0.15	0.11	0.04
c	0.06	0.02	0.02

Encontrar la correspondiente distribución de probabilidad p^1 en S . ¿Es una fuente sin memoria? ¿Cuál es la relación entre $H(p^2)$ y $H(p^1)$?

Ejercicio 5

Considérese una fuente que emite símbolos de un alfabeto $S = \{a, b, c\}$, con la probabilidad de distribución p^2 en S^2 definida por la siguiente tabla:

	a	b	c
a	0.39	0.17	0.04
b	0.15	0.11	0.04
c	0.06	0.02	0.02

Encontrar la correspondiente distribución de probabilidad p^1 en S . ¿Es una fuente sin memoria? ¿Cuál es la relación entre $H(p^2)$ y $H(p^1)$?

Solución:

$$p^1(a) = p^2(aa) + p^2(ab) + p^2(ac) = 0,6$$

$$p^1(b) = p^2(ba) + p^2(bb) + p^2(bc) = 0,3$$

$$p^1(c) = p^2(ca) + p^2(cb) + p^2(cc) = 0,1$$

La fuente no es sin memoria, ya que (por ejemplo) $p^2(aa) = 0,39$ mientras $p^1(a) \times p^1(a) = 0,36$

La entropía de p^1 is:

$$0,6 \times \log_2(1/0,6) + 0,3 \times \log_2(1/0,3) + 0,1 \times \log_2(1/0,1) \approx \mathbf{1.295}$$

La entropía de p^2 is:

$$0,39 \times \log_2(1/0,39) + 0,17 \times \log_2(1/0,17) + 0,04 \times \log_2(1/0,04) + 0,15 \times \log_2(1/0,15) + 0,11 \times \log_2(1/0,11) + 0,04 \times \log_2(1/0,04) + 0,06 \times \log_2(1/0,06) + 0,02 \times \log_2(1/0,02) + 0,02 \times \log_2(1/0,02) \approx \mathbf{2.566}$$

Por tanto se cumple:

$$H(p^2) < H(p^1) + H(p^1) = 2,566 < 1,295 + 1,295 = \mathbf{2.566} < \mathbf{2.59}$$

Ejercicio 6

Supongamos que una fuente estacionaria emite símbolos de un alfabeto $S' = \{a,b,c,d\}$ y la probabilidad de distribución p^2 es dada por la siguiente tabla:

	a	b	c	d
a	0.14	0.17	0.04	0.05
b	0.15	0.10	0.04	0.01
c	0.05	0.02	0.10	0.03
d	0.06	0.01	0.02	0.01

¿Cual es la distribución de probabilidad p^1 ? ¿Es una fuente sin memoria?

Ejercicio 6

Supongamos que una fuente estacionaria emite símbolos de un alfabeto $S' = \{a,b,c,d\}$ y la probabilidad de distribución p^2 es dada por la siguiente tabla:

	a	b	c	d
a	0.14	0.17	0.04	0.05
b	0.15	0.10	0.04	0.01
c	0.05	0.02	0.10	0.03
d	0.06	0.01	0.02	0.01

¿Cual es la distribución de probabilidad p^1 ? ¿Es una fuente sin memoria?

Solución:

$$p^1(a) = p^2(aa) + p^2(ab) + p^2(ac) + p^2(ad) = 0,4$$

$$p^1(b) = p^2(ba) + p^2(bb) + p^2(bc) + p^2(bd) = 0,3$$

$$p^1(c) = p^2(ca) + p^2(cb) + p^2(cc) + p^2(cd) = 0,2$$

$$p^1(d) = p^2(da) + p^2(db) + p^2(dc) + p^2(dd) = 0,1$$

La fuente no es sin memoria: $p^2(aa) = 0,14$ y $p^1(a) \times p^1(a) = 0,16$

Codificación de un código estacionario

- Establecemos el flujo emitido por una fuente como un flujo de bloques de longitud r .
- Si la fuente es estacionaria entonces, para cada $r \geq 1$, hay una distribución de probabilidad asociada p^r y su entropía $H(p^r)$. Esto representa la **incertidumbre del flujo**, por bloques de r símbolos.
- Para bloques de r símbolos, **la incertidumbre por símbolo** es $H(p^r)/r$, por lo que las incertidumbres, medidas en bits por símbolo, asociado con las distribuciones p^r ($r \geq 1$) son:

$$H(p^1), \frac{H(p^2)}{2}, \frac{H(p^3)}{3}, \dots, \frac{H(p^r)}{r}, \dots$$

Definición 3 (Entropía de una fuente estacionaria)

La **entropía H** de una fuente estacionaria con distribuciones de probabilidad p^r es el **ínfimo de los números**:

$$H = \inf_r \frac{H(p^r)}{r}$$

Esta definición es consistente con la definición de la entropía de una fuente sin memoria.

Ejemplo 4

Volviendo al ejercicio 5: Considérese una fuente que emite símbolos de un alfabeto $S = \{a, b, c\}$, con la probabilidad de distribución p^2 en S^2 definida por la siguiente tabla:

	a	b	c
a	0.39	0.17	0.04
b	0.15	0.11	0.04
c	0.06	0.02	0.02

Observamos la entropía de una fuente estacionaria $H(p^1)$, $H(p^2)/2$.

Solución:

$H(p^1)$ era **1.295** y $H(p^2)$ era: **2.566**.

Por tanto: $H(p^1) = \mathbf{1.295}$ y $\frac{H(p^2)}{2} = \mathbf{1.283}$

Teorema 2

Si una fuente estacionaria es sin memoria, su entropía H es igual a $H(p^1)$.

- En la práctica, solo es viable determinar $H(p^r)/r$ para algunos valores de r .
- En teoría es posible encontrar r por lo que la aproximación puede ser tan cercana como deseemos, y esto nos lleva a un resultado fundamental: es posible construir códigos con longitud media de palabra (en bits por símbolo) arbitrariamente cercana a H .

Teorema 3 (Teorema de codificación para fuentes estacionarias)

Suponemos que tenemos una fuente estacionaria que emite símbolos de un alfabeto S , con entropía H . Entonces, dado ($\varepsilon > 0$), existe un entero positivo n y un código binario sin prefijo para (S^n, p^n) para el cual la longitud media de palabra L_n satisface:

$$\frac{L_n}{n} < H + \varepsilon$$

En una fuente sin memoria: $L < H + \varepsilon$

Ejemplo 5

Volviendo al ejercicio 5: Considérese una fuente que emite símbolos de un alfabeto $S = \{a, b, c\}$, con la probabilidad de distribución p^2 en S^2 definida por la siguiente tabla:

	a	b	c
a	0.39	0.17	0.04
b	0.15	0.11	0.04
c	0.06	0.02	0.02

Solución:

Para la posible codificación de un símbolo: a:0, b:10, c:11

$$L_1 = 0,6 \times 1 + 0,3 \times 2 + 0,1 \times 2 = 1,40$$

Para la posible codificación de dos símbolos aa:0, bb:100, ba:101, ab:110

ac:11100, bc:11101, ca:11111, cb:111100, cc:111101

$L_2 = 2,62$ bits. Longitud media por símbolo: $L_2/2 = 1,31$ bits/símbolo Al aumentar n ,

L_n/n puede acercarse tanto como queramos a H .

Sección 4

Algoritmos para compresión de los datos

Algoritmos para compresión de los datos

- Supongamos que un archivo de N bits es producido por una fuente estacionaria con entropía H . Acorde con el teorema 3, codificando el archivo en bloques de suficiente longitud, podemos reducir el tamaño del archivo a $(H + \epsilon)N$ bits, donde ϵ es tan pequeño como deseemos.
- La regla de Huffman no encaja con este método en la práctica:
 - 1 Necesita de la construcción de las palabras codificadas para todos los bloques antes de que cualquier codificación pueda ser realizada, y si la longitud de bloque requerida es n , el conjunto de palabras codificadas tendrá longitud 2^n .
 - 2 Asume que las características de la fuente son conocidas por anticipado y a menudo podríamos desear actualizar las probabilidades durante el procesamiento.
- Por tanto existe una necesidad para la creación de métodos alternativos para codificar con las siguientes propiedades:
 - 1 las palabras codificadas para un cierto bloque pueden ser calculadas de forma aislada, sin la necesidad de establecer todas las palabras codificadas.
 - 2 las palabras codificadas puede ser actualizadas in situ (esto es conocido como codificación adaptativa)

En las siguiente secciones describimos dos métodos:

- 1 **Codificación aritmética**, es basado en la idea de asignar palabras codificadas cortas a símbolos que tienen asignadas altas frecuencias.
- 2 **Codificación de diccionario**, es basada en una solución más heurística, pero sin embargo ha funcionado bien en la práctica.

Sección 4.1

Codificación Aritmética

Utilizar números como palabras codificadas

- La técnica conocida como codificación aritmética es basada en la correspondencia entre palabras binarias (elementos de $\mathbb{B}^*$ y fracciones (elementos de el conjunto $\mathbb{Q}$ de números racionales).
- Para cualquier palabra $z_1 z_2 \dots z_n$ en $\mathbb{B}^n$. le corresponde un número racional

$$z_1/2 + z_2/2^2 + \dots z_n/2^n$$

- El número estará entre 0 y 1, y es denotado como $0.z_1 z_2 \dots z_n$ en notación binaria.
- Por lo que para cada entero $n \geq 1$ tendremos una función $\mathbb{B}^n \rightarrow \mathbb{Q}$, definida como:

$$z_1 z_2 \dots z_n \mapsto 0.z_1 z_2 \dots z_n$$

Ejemplo 6

Establece los número racionales correspondientes a las palabras 0110101 y 1010000.

Ejemplo 6

Establece los número racionales correspondientes a las palabras 0110101 y 1010000.

Solución:

En este caso $n = 7$. La palabra 0110101 corresponde al número racional

$$0.0110101 = 1/2^2 + 1/2^3 + 1/2^5 + 1/2^7 = 1/4 + 1/8 + 1/32 + 1/128 = 53/128,$$

o 0.4140625 en notación usual (decimal).

Similarmente, la palabra 1010000 corresponde al número racional

$$0.1010000 = 1/2^1 + 1/2^3 = 1/2 + 1/8 = 5/8,$$

o 0.625 en notación decimal.

Ejercicio 7

Establezca los números racionales correspondientes a las palabras 0111010 y 1001001.

Ejercicio 7

Establezca los números racionales correspondientes a las palabras 0111010 y 1001001.

Solución:

En este caso $n = 7$. La palabra 0111010 corresponde al número racional

$$0.0111010 = 1/2^2 + 1/2^3 + 1/2^4 + 1/2^6 = 1/4 + 1/8 + 1/16 + 1/64 = (16 + 8 + 4 + 1) / 64 = 29 / 64,$$

La palabra 1001001 corresponde al número racional

$$0.1001001 = 1/2^1 + 1/2^4 + 1/2^7 = 1/2 + 1/16 + 1/128 = (64 + 8 + 1) / 128 = 73 / 128,$$

- En la **codificación aritmética** utilizamos los números racionales para definir las **palabras codificadas** que representan las cadenas de símbolos.
- Como hemos venido haciendo debemos asegurar que la **cadena X con una probabilidad alta** es representada por la palabra codificada $c(X)$ con **menor longitud n**.
- Comenzamos con el conjunto $\mathbb{R}$ de número reales, debido a que la **probabilidad de un evento** esta asociada a un número real, más que a un número racional.
- Deberemos escoger la palabra codificada $c(X)$ que sea un número en un intervalo apropiado de la forma

$$[a, a + P) = \{r \in \mathbb{R} | a \leq r < a + P\},$$

donde a es un número real y P es la probabilidad de la cadena X .

Teorema 4

Supongase a y P tal que $0 \leq a < a + P \leq 1$. Sea n cualquier entero tal que $2^n > 1/P$. Entonces existe una palabra $z_1 z_2 \dots z_n \in \mathbb{B}^n$ tal que

$$0.z_1 z_2 \dots z_n \in [a, a + P).$$

La cadena X con una probabilidad alta es representada por la palabra codificada $c(X)$ con menor longitud n . Con esta desigualdad $2^n > 1/P$ tenemos que si P crece n disminuye. Por ejemplo:

Mensaje	P	Intervalo $[a, a + P)$	$1/P$	Condición $2^n > 1/P$
A	0.5	$[0,0, 0,5)$	2	$2^2 > 2$
B	0.25	$[0,5, 0,75)$	4	$2^3 > 4$
C	0.125	$[0,75, 0,875)$	8	$2^4 > 8$

Cada mensaje X ocupa un intervalo de longitud $P(X)$ dentro de $[0,1)$

- Si $P(X)$ es grande, el intervalo $[a, a+P)$ es ancho, y un número binario de pocos bits ya “cabe” dentro de él.
- Si $P(X)$ es pequeño, el intervalo es estrecho, y necesitas más bits (mayor precisión) para que el número binario caiga dentro del intervalo.

Codificación Aritmética

- Utilizaremos el teorema 4 para construir códigos binarios sin prefijo para las cadenas de símbolos que son emitidos por una fuente estacionaria.
- En este caso hay una probabilidad de distribución en el conjunto S^r de cadenas $X = x_1x_2 \dots x_r$ de longitud r en S . **En general no asumimos la propiedad de independencia:**

$P(x_1x_2 \dots x_r)$ no es necesariamente igual a $P(x_1)P(x_2) \dots P(x_r)$.

- Podemos suponer que los símbolos en el alfabeto S están establecido en un **orden específico:**

$$\alpha < \beta < \gamma < \dots < \omega$$

- ▶ Utilizando este orden definimos un orden para cadenas de símbolos, de la misma forma las **palabras del diccionario** son ordenados utilizando orden alfabético de las letras.

Definición 4 (Orden Diccionario)

El orden de diccionario en S^r es definido como sigue:

- Si X, Y son dos cadenas diferentes en S^r , sea i el menor entero tal que $x_i \neq y_i$.
 - ▶ Si $y_i < x_i$ establecemos $Y < X$,
 - ▶ de otra forma $X < Y$.
- Es conveniente reservar α y ω para el primer y último símbolo, independientemente del tamaño de S .
- Por ejemplo, si $|S| = 3$, establecemos $S = \{\alpha, \beta, \omega\}$, con el orden $\alpha < \beta < \omega$.
- En este caso el número de cadenas de longitud 4 es $3^4 = 81$. Las primeras siete cadenas son
$$\alpha\alpha\alpha\alpha < \alpha\alpha\alpha\beta < \alpha\alpha\alpha\omega < \alpha\alpha\beta\alpha < \alpha\alpha\beta\beta < \alpha\alpha\beta\omega < \alpha\alpha\omega\alpha < \dots$$
- y las últimas siete cadena son
$$\dots < \omega\omega\alpha\omega < \omega\omega\beta\alpha < \omega\omega\beta\beta < \omega\omega\beta\omega < \omega\omega\omega\alpha < \omega\omega\omega\beta < \omega\omega\omega\omega.$$

Definición 5 (Función de probabilidad acumulativa)

Dada una distribución P en S^r definimos la función de probabilidad acumulativa a en S^r como sigue.

- *Si X es la primera cadena en S^r establecemos $a(X) = 0$.*
- *De otra forma*

$$a(X) = \sum_{Y < X} P(Y)$$

Por ejemplo, supongamos que $S = \{\alpha, \beta, \gamma, \omega\}$ y tomamos $r = 1$. Si los valores de P en S son

$$P(\alpha) = 0,5, P(\beta) = 0,3, P(\gamma) = 0,1, P(\omega) = 0,1,$$

entonces los valores de a son:

$$a(\alpha) = 0,0, a(\beta) = 0,5, a(\gamma) = 0,8, a(\omega) = 0,9.$$

Notese que, para cada $x \in S$ hay un intervalo correspondiente $[a(x), a(x) + P(x))$, y estos intervalos forman una partición del intervalo $[0, 1)$.

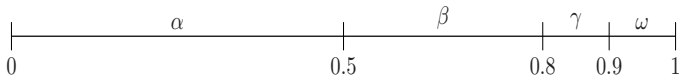


Figura 1: La partición de $[0, 1)$ asociada con una distribución dada en S^1

Generalmente, dada una cadena $X \in S^r$ denotamos por I_X el correspondiente intervalo:

$$I_X = [a(X), a(X) + P(X)).$$

Para cada $r \geq 1$ estos intervalos forman una partición de $[0, 1)$.

Definición 6 (Código Aritmético)

- Supongamos que un código estacionario emite símbolos de un conjunto S y la probabilidad de distribución en S^r es conocido.
- Para $X \in S^r$ sea $P = P(X)$, definimos n_P el menor entero tal que $2^{n_P} \geq 1/P$, y sea $n = n_P + 1$.
- La codificación aritmética $c : S^r \rightarrow B^*$ es definida tomando que $c(X)$ sea la palabra $z_1 z_2 \dots z_n$ que representa el único entero c para el cual $c - 1 \leq 2^n a(X) < c$.
- Según el teorema 4, la condición $n = n_P + 1$ asegura que el número $0.z_1 z_2 \dots z_n$ esta en el intervalo I_X .

Ejemplo 7

Sean $S = \{\alpha, \beta, \gamma, \omega\}$. La siguiente tabla define una probabilidad de distribución en S^2 : por ejemplo, el número en la fila β y la columna γ es la probabilidad de $\beta\gamma$

	α	β	γ	ω
α	0.31	0.13	0.04	0.02
β	0.13	0.15	0.01	0.01
γ	0.04	0.01	0.03	0.02
ω	0.02	0.01	0.02	0.05

Determina el código aritmético para S^2 .

Solución:

- Para cada cadena $X \in S^2$ tomaremos $P = P(X)$, $a = a(X)$ y calcularemos $1/P$, $n_P \mid 2^{n_P} \geq 1/P$ y $n = n_P + 1$.
- Después determinaremos c que cumpla que $c - 1 \leq 2^n a < c$, y establecemos $c(X)$ la palabra binaria de longitud n correspondiente a c .
- Los cálculos para las primeras palabras codificadas son tabuladas a continuación. Notese que cada fracción $c/2^n$ está incluido en el intervalo $[a, a + P)$, como garantiza el teorema 3.

X	P	a	1/P	n_P	n	c	c(X)
$\alpha\alpha$	0.31	0.00	3.2	2	3	1	001
$\alpha\beta$	0.13	0.31	7.7	3	4	5	0101
$\alpha\gamma$	0.04	0.44	25	5	6	29	011101
$\alpha\omega$	0.02	0.48	50	6	7	62	0111110
$\beta\alpha$	0.13	0.50	7.7	3	4	9	1001
...	...	...	...	...	...	...	...

Aunque hemos seguido un ejemplo **no es necesario calcular las palabras codificadas en orden.**

Por ejemplo, si necesitamos la palabra codificada para $\gamma\beta$ podemos calcular directamente, conociendo solamente que $P(\gamma\beta) = 0,01$ y $a(\gamma\beta) = 0,84$.

X	P	a	1/P	n_P	n	c	c(X)
$\gamma\beta$	0.01	0.84	100	7	8	216	11011000

Si quisiera decodificar una palabra, por ejemplo 0111110, entonces:

- 1 Convierte $0,0111110_2$ a decimal:
 $0,0111110_2 = 1/4 + 1/8 + 1/16 + 1/32 + 1/64 = 0,484375$.
- 2 Busca el intervalo en la tabla donde esté 0,484375.
 $\alpha\omega : a = 0,48, P = 0,02 \Rightarrow [a, a + P) = [0,48, 0,50)$.
- 3 Comprueba pertenencia: $0,484375 \in [0,48, 0,50)$.

Nótese que no es una fuente sin memoria, ya que (por ejemplo)

$$P(\alpha) = P(\alpha\alpha) + P(\alpha\beta) + P(\alpha\gamma) + P(\alpha\omega) = 0.31 + 0.13 + 0.04 + 0.02 = 0.5, \text{ y}$$
$$P(\alpha\alpha) = 0.31 \neq P(\alpha) \times P(\alpha).$$

Ejercicio 8

Encuentra la palabra codificada $\omega\alpha$ para el código aritmético en la distribución del anterior:

Ejercicio 8

Encuentra la palabra codificada $\omega\alpha$ para el código aritmético en la distribución del anterior:

Solución:

X	P	a	1/P	n_P	n	c	c(X)
$\omega\alpha$	0.02	0.90	50	6	7	116	1110100

Ejemplo 8

Programas de ejemplo en python:

- ➊ syntheticCodev2.py:
 - ▶ Generar textos sintéticos con alfabetos pequeños y control de entropía.
- ➋ block arith codes.py:
 - ▶ Construir el modelo de probabilidades de **datagramas** .
 - ▶ Codificar cada datagrama $X \in S^r$ con una palabra binaria $c(X)$.
 - ▶ Medir tamaño comprimido, *bits por carácter* (bpc) y *ratio* comprimido/original.

Generador: syntheticCodev2.py

Uso:

```
# Maxima entropia (uniforme)
python3 syntheticCodev2.py --preset max -o uniform_20k.txt --show-stats

# Entropia media (por defecto 0.40,0.25,0.20,0.10,0.05)
python3 syntheticCodev2.py --preset mid -o mid_20k.txt --show-stats

# Minima entropia (muy sesgada)
python3 syntheticCodev2.py --preset min -o min_20k.txt --show-stats

# Probabilidades personalizadas
python3 syntheticCodev2.py --alphabet ABCDE \
    --probs 0.35,0.30,0.20,0.10,0.05 -n 30000 -o custom_30k.txt --show-
stats
```

Generador: syntheticCodev2.py

Ejemplo salida:

```
python3 syntheticCodev2.py --preset max -o uniform_20k.txt --show-stats
Generado uniform_20k.txt con 20000 simbolos.
- Configuracion -
Alfabeto: ABCDE
Probabilidades teoricas:
  A: 0.200000 (20.00%)
  B: 0.200000 (20.00%)
  C: 0.200000 (20.00%)
  D: 0.200000 (20.00%)
  E: 0.200000 (20.00%)
Entropia teorica  $H = 2.3219$  bits/simbolo
Frecuencias observadas:
  A: 3911 ( 19.55%)
  B: 4025 ( 20.12%)
  C: 4092 ( 20.46%)
  D: 3999 ( 19.99%)
  E: 3973 ( 19.86%)
Entropia observada  $H_{obs} = 2.3218$  bits/simbolo
```

Programa de codificación: block_arith_codes.py

Codificación:

```
python3 block_arith_codes.py encode mid_20k.txt --ngram 3 --show-table
```

Salidas:

- .barc: flujo binario (concatenación de los $c(X)$).
- .model.json: tabla con $X, P, a, 1/P, n, c, c(X)$ y cola (tail).
- Métricas: tamaño original vs. comprimido, ratio, bpc, bits/token.

Decodificación:

```
python3 block_arith_codes.py decode mid_20k.txt.ng3.barc \  
--model mid_20k.txt.ng3.barc.model.json -o decoded.txt
```

Ejemplo de tabla generada

X	P	a	$1/P$	n_P	n	c	$c(X)$
AA	0.310000	0.000000	3.2	2	3	1	001
AB	0.130000	0.310000	7.7	3	4	5	0101
AC	0.040000	0.440000	25.0	5	6	29	011101
AW	0.020000	0.480000	50.0	6	7	62	0111110
BA	0.130000	0.500000	7.7	3	4	9	1001

Demostración en vivo (sugerencias)

1) Generar datos

```
python3 syntheticCodev2.py --preset mid -o mid_20k.txt --show-stats
```

2) Codificar con distintos bloques

```
python3 block_arith_codes.py encode mid_20k.txt --ngram 1
```

```
python3 block_arith_codes.py encode mid_20k.txt --ngram 2
```

```
python3 block_arith_codes.py encode mid_20k.txt --ngram 3 --show-table  
--table-digits 8
```

3) Decodificar y verificar

```
python3 block_arith_codes.py decode mid_20k.txt.ng3.barc \  
--model mid_20k.txt.ng3.barc.model.json -o mid_20k_decoded.txt
```


Ejercicio 9

Establezca la longitud media L_2 para el código anterior. (No es necesario calcular las palabras codificadas). Compare los valores de $L_2/2$, $H(p^1)$, $H(p^2)/2$

Solución:

X	P	a	1/P	n_P
$\alpha\alpha$	0.31	0.00	3.2	2
$\alpha\beta$	0.13	0.31	7.7	3
$\alpha\gamma$	0.04	0.44	25	5
$\alpha\omega$	0.02	0.48	50	6
$\beta\alpha$	0.13	0.50	7.7	3
$\beta\beta$	0.15	0.63	6.6	3
$\beta\gamma$	0.01	0.78	100	7
$\beta\omega$	0.01	0.79	100	7
$\gamma\alpha$	0.04	0.80	25	5
$\gamma\beta$	0.01	0.84	100	7
$\gamma\gamma$	0.03	0.85	33,33	6
$\gamma\omega$	0.02	0.88	50	6
$\omega\alpha$	0.02	0.90	50	6
$\omega\beta$	0.01	0.92	100	7
$\omega\gamma$	0.02	0.93	50	6
$\omega\omega$	0.05	0.95	20	5

$$L_2 = 1 + \sum p_i \times n_p = 2 * 0,31 + 3 * 0,13 + 5 * 0,04 + 6 * 0,02 + 3 * 0,13 + 3 * 0,15 + 7 * 0,01 + 7 * 0,01 + 5 * 0,04 + 7 * 0,01 + 6 * 0,03 + 6 * 0,02 + 6 * 0,02 + 7 * 0,01 + 6 * 0,02 + 5 * 0,05 = 1 + 3,44 = 4,44.$$

$$L_2/2 = 2.22.$$

$$H(p^1) = 1.685.$$

$$H(p^2)/2 = 1.495.$$

Ejercicio 10

Construya el código aritmético para la distribución de probabilidad en S^2 dado en el ejemplo 5, tomando $a < b < c$. Compare los valores de $L_2/2$, $H(p^1)$, $H(p^2)/2$. [No es necesario establecer las palabras codificadas]

	a	b	c
a	0.39	0.17	0.04
b	0.15	0.11	0.04
c	0.06	0.02	0.02

Solución:

X	P	a	1/P	n_P	n	c	c(X)
aa	0.39	0.00	2.5	2	3	1	001
ab	0.17	0.39	5.8	3	4	7	0111
ac	0.04	0.56	25	5	6	36	100100
ba	0.15	0.60	6.6	3	4	10	1010
bb	0.11	0.75	9.1	4	5	25	11001
bc	0.04	0.86	25	5	6	56	111000
ca	0.06	0.90	16.6	5	6	58	111010
cb	0.02	0.96	50	6	7	123	1111011
cc	0.02	0.98	50	6	7	126	1111110

$$L_2 = \sum p_i * n = 1 + \sum p_i * n_p = 3 * 0,39 + 4 * 0,17 + 6 * 0,04 + 4 * 0,15 + 5 * 0,11 + 6 * 0,04 + 6 * 0,06 + 7 * 0,02 + 7 * 0,02 = 4,12.$$

$$L_2/2 = 2.06$$

$$H(p^1) = 1,295.$$

$$H(p^2) = 2,566;$$

$$H(p^2)/2 = 1,283$$

- Hemos mostrado en la codificación aritmética que pueden ser contruidos de forma aislada, por lo que en la práctica es más eficiente que utilizar la regla de Huffman.
- Nos queda establecer que la codificación aritmética tiene las propiedades deseables:
 - ▶ (i) el código es libre de prefijo, por lo que la codificación puede ser realizada 'en línea', y
 - ▶ (ii) el código es cercano al óptimo. Estos hechos son consecuencia de la elección $n = n_P + 1$ en la definición y garantizar que $c/2^n \in [a, a + P)$.

Teorema 5

Un código aritmético construido acorde la definición 6 es libre de prefijo.

Teorema 6

Sea P una probabilidad en el conjunto S^r de cadenas de longitud r , y sea L la longitud media de palabra del correspondiente código aritmético. Entonces

$$L < H(P) + 2.$$

Sección 4.2

Codificación con un diccionario dinámico

Codificación con un diccionario dinámico

Definición 7 (Diccionario, índice)

Un diccionario D basado en el alfabeto S es una secuencia de palabras distintas en S^* :

$$D = d_1, d_2, d_3, \dots, d_N$$

Además establecemos que el índice de d_i es i .

- En la codificación aritmética impusimos un “orden alfabetico” en el conjunto de símbolos S , y construimos el correspondiente “orden diccionario” en S^r .
- Por ejemplo, si S tiene tres símbolos en el orden $\alpha < \beta < \gamma$, el conjunto S^2 es un diccionario con el orden

$$\alpha\alpha, \alpha\beta, \alpha\gamma, \beta\alpha, \beta\beta, \beta\gamma, \gamma\alpha, \gamma\beta, \gamma\gamma$$

- y el índice del elemento $\gamma\alpha$ es 7.
- Este es un ejemplo de diccionario estático.

- En esta sección consideramos un método de codificación en el cual el diccionario es dinámico, en otras palabras es construido durante el proceso, por lo que contiene aquellas cadenas que ocurren en un mensaje dado.
- Asumimos que la fuente es estacionaria por lo que las cadenas de símbolos que suceden en el comienzo del mensaje son las típicas que ocurrirán mas adelante, por lo que no necesitamos información específica sobre la distribución de los símbolos.
- Describimos el sistema de codificación Lempel-Ziv-Welch, también conocido como LZW.

- El alfabeto S tiene el tamaño m , con símbolos $s_1 < s_2 < \dots < s_m$. Tanto codificador como decodificador comienzan con el diccionario

$$D_0 = d_1, d_2, \dots, d_m, \text{ donde } d_i = s_i \text{ (} i = 1, 2, \dots, m \text{)}.$$

- Si el mensaje es $X = x_1 x_2 \dots x_n$, donde $x_i \in S$ ($i = 1, 2, \dots, m$),
- El codificador hace una codificación del mensaje, $c(X)$ y al mismo tiempo construye un diccionario D_X añadiendo a D_0 ciertas cadenas $s_p s_q \dots$ que ocurre en X .
- La regla de codificación reemplaza cada cadena por su índice en D_X .
- El decodificador utiliza la secuencia de números $c(X)$, y el diccionario inicial D_0 , para reconstruir X y D_X .

Definición 8 (Codificación LZW)

Supongase que un mensaje $X = x_1x_2 \dots x_n$ en el alfabeto $S = \{s_1, s_2, \dots, s_m\}$ dado. Sea $D_0 = d_1, d_2, \dots, d_m$, donde $d_i = s_i$ ($i = 1, 2, \dots, m$).

La reglas de codificación LZW construye $c(X) = c_1c_2c_3 \dots$ en una serie de pasos.

- Paso 1. El primer símbolo x_1 ($= s_p$) es una entrada d_p en D_0 .

- 1 Codificamos x_1 definiendo $c_1 = p$.

- 2 La cadena $x_1x_2 (= s_ps_q)$ no esta en D_0 . Definimos

$$d_{m+1} = x_1x_2, D_1 = (D_0, d_{m+1}).$$

- Continua...

Definición 9 (Codificación LZW (Continuación))

- Paso k ($k \geq 2$) Supongase que hemos completado los pasos $1, 2, \dots, k-1$. Esto significa que el código $c_1 c_2 \dots c_{k-1}$ para un segmento inicial $x_1 x_2 \dots x_i$ de X , y además hemos construido un diccionario $D_{k-1} = d_1, d_2, \dots, d_{m+k-1}$. Encontrar la cadena más larga w de la forma

$$w = x_{i+1} \dots x_j (j \geq i + 1)$$

tal que w está en D_{k-1} , sea $w = d_t$. Tal cadena existe, debido a que x_{i+1} es un solo símbolo y ya está incluido en el diccionario inicial D_0 . Por definición, $w x_{j+1}$ no está en D_{k-1} . Codificamos el segmento $x_{i+1} \dots x_j$ de X mediante

$$c_k = t, \text{ y define } d_{m+k} = w x_{j+1}, D_k = (D_{k-1}, d_{m+k}).$$

- Repetir el procedimiento hasta alcanza el final del mensaje.

Teorema 7

Un código LZW construido siguiendo la Definición 8 es unívocamente decodificable.

Ejemplo 9

Siendo el alfabeto $S = \{A, B, C, D, R\}$ y aplicar las reglas de codificación LZW al mensaje

$X = \text{ABRACADABRA}$

Ejemplo 9

Siendo el alfabeto $S = \{A, B, C, D, R\}$ y aplicar las reglas de codificación LZW al mensaje

$$X = \text{ABRACADABRA}$$

Solución:

Comenzamos con el diccionario $D_0 = A, B, C, D, R$. La regla para el Paso 1, nos indica que codifiquemos A , el cual tiene índice 1 y añadimos AB al diccionario:

$$c(A) = 1, D_1 = A, B, C, D, R, AB.$$

En el paso 2, B está en D_1 pero no BR , por lo que codificamos B por su índice 2 y añadimos BR :

$$c(AB) = 1\ 2, D_2 = A, B, C, D, R, AB, BR.$$

Las reglas dicen que en los Pasos 3,4,5,6,7 continuamos de una forma similar hasta que alcanzamos el estado:

$$c(\text{ABRACAD}) = 1\ 2\ 5\ 1\ 3\ 1\ 4,$$
$$D_7 = A,B,C,D,R,AB,BR,RA,AC,CA,AD,DA.$$

En el Paso 8 AB ya está en D_7 , con índice 6, pero ABR no está en D_7 , por lo que el estado es

$$c(\text{ABRACADAB}) = 1\ 2\ 5\ 1\ 3\ 1\ 4\ 6,$$
$$D_8 = A,B,C,D,R,AB,BR,RA,AC,CA,AD,DA,ABR.$$

En el Paso 9, RA está en el diccionario, con índice 8, y el mensaje finaliza, por lo que

$$c(\text{ABRACADABRA}) = 1\ 2\ 5\ 1\ 3\ 1\ 4\ 6\ 8.$$

Ejemplo 10

Dado el diccionario $D_0 = \text{I,M,P,S}$, decodifica el mensaje

214468331

Ejemplo 10

Dado el diccionario $D_0 = I, M, P, S$, decodifica el mensaje

214468331

Solución:

En el Paso 1, $c_1 = 2$ y $d_2 = M$ por lo que $x_1 = M$. También $c_2 = 1$ y $d_1 = I$ por lo que la nueva entrada en el diccionario es MI .

$$2 \mapsto M, D_1 = I, M, P, S, MI$$

En el Paso 2, $c_2 = 1$ y $d_1 = I$ por lo que $x_2 = I$. Además $c_3 = 4$ y $d_4 = S$, por tanto la nueva entrada en el diccionario es IS . Por tanto,

$$21 \mapsto MI, D_1 = I, M, P, S, MI, IS$$

Los Pasos 3 y 4 son similares, y tienen como resultado

$$2144 \mapsto MISS, D_4 = I, M, P, S, MI, IS, SS, SI$$

En el siguiente paso $c_5 = 6$, y $d_6 = IS$ por lo que $x_5 = I, x_6 = S$. También $c_6 = 8$ y $d_8 = SI$ por tanto la entrada del diccionario es ISS . Por tanto,

$$21446 \mapsto MISSIS, D_5 = I, M, P, S, MI, IS, SS, SI, ISS$$

Pasos 7,8,9 son similares, y el resultado es

$$214468331 \mapsto MISSISSIPPI$$

- En este punto, podemos plantearnos porqué las reglas LZW alcanzan una compresión importante.
- En estos ejemplos, solo alcanzamos una pequeña cantidad de compresión con el costo asociado.
- Por ejemplo, el mensaje *ABRACADABRA* tiene longitud 11, y la codificación 125131468 tiene una longitud de 9.
- Sin embargo en la práctica cuando el mensaje incrementa en longitud, con el inevitable incremento de cadenas, entonces se produce una mejora.
- En la práctica, LZW ofrece unos buenos resultados.

Ejemplo 11

Programa de ejemplo en python:

① lzw.py:

- ▶ Construir el **diccionario inicial** con los símbolos únicos del texto (alfabeto).
- ▶ Recorrer el texto añadiendo nuevas entradas $w + k$ al diccionario cada vez que no existan.
- ▶ Emitir el código correspondiente a cada cadena w encontrada en el diccionario.
- ▶ Generar el fichero comprimido con cabecera (LZW1), alfabeto inicial y secuencia de códigos.

Generador: lzw.py

Uso:

```
# Codificar con tabla de pasos
python3 lzw.py encode uniform_20k.txt --show-steps --colw 30 -o
    uniform_20k.lzw --output-json uniform_20k.meta.json

# Inspeccionar el binario
python3 lzw.py inspect uniform_20k.lzw

# Decodificar
python3 lzw.py decode uniform_20k.lzw -o uniform_20k.out.txt
```

Generador: lzw.py

i	w	k	emit	add to dict
1	E	E		
2	E	C	0	EC
3	C	A	1	CA
4	A	E	2	AE
5	E	E	0	EE
6	EC	C		
7	EC	D	5	ECD
8	D	A	3	DA
9	A	D	2	AD
10	D	B	3	DB

i -- > Número de paso (iteración del algoritmo)

w -- > Cadena actual (la más larga encontrada en el diccionario hasta ahora)

k -- > Siguiete símbolo leído del texto

emit -- > Código numérico emitido al flujo de salida

add to dict -- > Nueva entrada añadida al diccionario (concatenación $w+k$)

Output: uniform_20k.lzw

```
4c5a 5731 0500 0000 0100 0000 4501 0000
0043 0100 0000 4101 0000 0044 0100 0000
429e 1200 0000 0000 0000 0000 0001 0000
0002 0000 0000 0000 0005 0000 0003 0000
0002 0000 0003 0000 0004 0000 000b 0000
```

4c 5a 57 31 → ASCII "LZW1"

05 00 00 00 → m = 5 símbolos en el alfabeto

0100 0000 45 → len=1, símbolo 'E'

0100 0000 43 → len=1, símbolo 'C'

0100 0000 41 → len=1, símbolo 'A'

0100 0000 44 → len=1, símbolo 'D'

0100 0000 42 → len=1, símbolo 'B'

9e 1200 0000 0000 0000 0000 → 4766 códigos

0001 0000 → símbolo 1

0002 0000 → símbolo 2

0000 0000 → símbolo 0

0005 0000 → símbolo 5

0003 0000 → símbolo 3

José A. Montenegro Montes
Dpto. Lenguajes y Ciencias de la Computación
ETSI Informática. Universidad de Málaga
jmmontes@uma.es

